Ditsha Live-System Verification Guide

How to prove the Review 2 changes by interacting with the running systems

Purpose. This guide demonstrates the claims through the citizen app, officer dashboard, API, ML service, and PostgreSQL database. It is evidence from real interactions, not only source-code inspection.

1. Start the real systems

Open four PowerShell terminals and run the following commands.

```
Terminal 1 - database and ML service
cd C:\Users\FCSE24-026\Desktop\ditsha-app
docker compose up -d db ml

Terminal 2 - API
cd C:\Users\FCSE24-026\Desktop\ditsha-app\api
npm run dev

Terminal 3 - officer dashboard
cd C:\Users\FCSE24-026\Desktop\ditsha-app\ditsha-frontend
npm run dev -- --port 5177

Terminal 4 - citizen application
cd C:\Users\FCSE24-026\Desktop\ditsha-app\ditsha-flutter
flutter run -d web-server --web-port 8081
```

Open these URLs:

System	URL
Officer dashboard	http://localhost:5177
Citizen app	http://localhost:8081
API health	http://localhost:3000/api/health
ML health	http://localhost:8000/health

```
curl.exe http://localhost:3000/api/health
curl.exe http://localhost:8000/health
```

2. Demonstrate the citizen workflow

Sign in with the seeded QA account:

```
Email:    citizen@example.com
Password: CitizenPass123!
```

A. Create a draft

Select a Botswana land board, choose an application type, enter a description, and save the application as a draft. Show the draft in the citizen app.

B. Upload a document

Upload a real PDF or image. Then try a renamed text file such as fake.pdf. The system must reject it because the file signature does not match the claimed type.

C. Submit

Submit the draft and show the status changing from draft to submitted.

D. Request information and respond

Leave the citizen app open. From the officer dashboard, request more information. Refresh the citizen app, respond to the request, and upload the missing document. Show the application returning to under review.

E. Withdraw

Use another active application and withdraw it. Show the status becoming withdrawn.

3. Demonstrate the officer workflow

Open http://localhost:5177 and sign in:

```
Email:    officer@example.com
Password: OfficerPass123!
```

Show the officer queue, search/filter, application details, timeline, documents, notes, request-information action, assignment/reassignment, ML assistance, and approval/rejection. The UI should reflect the citizen actions performed in the previous section.

4. Prove the database records the live interactions

Run these queries after the demonstration. They show that the UI actions created real database rows.

Documents: versioning, checksums, replacement, and soft deletion

```
docker exec -it ditsha-backend-db-1 psql -U postgres -d ditsha -c "SELECT document_id, application_id, original_name, document_
```

Predictions stored by the ML assistance flow

```
docker exec -it ditsha-backend-db-1 psql -U postgres -d ditsha -c "SELECT prediction_id, application_id, category, score, prio
```

Workflow timeline

```
docker exec -it ditsha-backend-db-1 psql -U postgres -d ditsha -c "SELECT application_id, event_type, from_status, to_status, 
```

Information requests

```
docker exec -it ditsha-backend-db-1 psql -U postgres -d ditsha -c "SELECT request_id, application_id, message, status, request
```

Prove ml_prediction replaced the proposed RISK_SCORE table

```
docker exec -it ditsha-backend-db-1 psql -U postgres -d ditsha -c "SELECT to_regclass('public.application_document') AS applica
```

Expected: application_document and ml_prediction contain their table names; old_risk_score is blank.

5. Call the live ML service

```
$mlRequest = @{
    application_id = "live-demo-001"
    application_type = "residential_allocation"
    description = "This is a sufficiently detailed application description for testing."
    document_names = @("identity_document")
    required_documents = @("identity_document", "proof_of_residence")
    duplicate_active_count = 0
    land_board_workload = 50
} | ConvertTo-Json

Invoke-RestMethod -Uri http://localhost:8000/v1/assist -Method Post -ContentType "application/json" -Body $mlRequest | Convert
```

The response should identify the missing proof of residence, include a model version, and show deterministic=true. This proves the running ML service is advisory and not an automatic land decision-maker.

6. Prove the predictive-model correction

Run the historical completeness experiment and the corrected delay experiment separately:

```
cd C:\Users\FCSE24-026\Desktop\ditsha-app\ml-service
python train_compare.py
python train_delay.py
python -m unittest -v
```

Historical artifact: comparison.json reports target label_incomplete with perfect scores. Explain that this was leakage because the label was calculated from the same completeness features supplied to the model.

Corrected artifact: comparison_delay.json reports target label_delayed, a 30-day delay threshold, and explicitly excludes processing_days and label_delayed from model features. The corrected results are realistic rather than perfect (approximately 0.74 accuracy and 0.80 F1 on the synthetic test split).

7. Suggested review sequence

1. Start database, ML service, API, dashboard, and citizen app.
2. Create and save a citizen draft.
3. Upload a valid document and reject an invalid fake PDF.
4. Submit the application.
5. Request information from the officer dashboard.
6. Respond as the citizen and upload the requested document.

7. Assign the application and request ML assistance.

8. Approve or reject the application.

9. Run the PostgreSQL queries and show the created rows.

10. Call the live ML endpoint.

11. Run both ML comparison scripts and explain the leakage correction.

Conclusion to state: ?I am proving the changes through live interaction: a real citizen request, real document validation, a real officer information loop, a real ML-service response, and the PostgreSQL rows created by those actions.?

Repository evidence: commit db2653a99fbfa440022c2641a29294e7d740b514